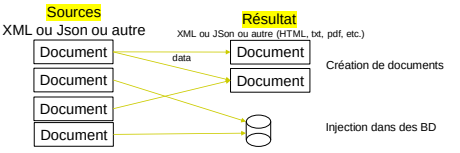
Transformer des documents structurés XML ou JSon

Transformer des documents structurés XML ou JSon

Pour quoi faire?

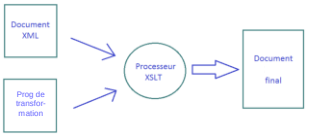


Principe de la transformation

- Entrée(s) : un(des) document(s) XML ou Json
- Opération de transformation
- Sortie : un document (XML, Json, ou autre) contenant des info issues des entrées et d'autres info

Principe

- Utilisation d'un Processeur (ex: XSLT)



- Codage par programme



- Avantages et inconvénients

Exemple de transformation d'un XML: feuille de style XSL

Présenter un document XML dans un navigateur
→ Transformer le doc XML en un doc HTML

- Une feuille de style .xsl (c'est un programme XSLT)

```
<? xml version = "1.0" encoding = "utf-8"?>
<xsl:stylesheet version="1.0"  xmlns:xsl="http://www.w3.org/1999/XSL/Transform">
```

puis début du programme de transformation XSLT…

- Dans le fichier XML, appel à la feuille de style

```
<? xml-stylesheet type="text/xsl" href="LeFichier.xsl"?>
```

puis le contenu du fichier XML…

```
<baseClient> <Client ID="01"> Dupont </Client>
…
</baseClient>
```


Pour Json: faut-il utiliser XSLT (ou équivalent)?

XSLT ou autres propositions: JSLT, etc. (17+ propositions sur Stackoverflow, et discussions)

Mais JSon → accès immédiat aux objets représentés

L'option 1 (modèle de transformation) est vu comme "système supplémentaire"! Donc peu utilisé.

Choix option 2: Transformations via le langage de programmation des objets (Java for Java objects, etc: Perl, Python, c#, PHP) et un peu d'algorithmie!

Les principes d'un modèle de transformation

- Un langage de règles de transformation
- Fonctions supportées
  - Lecture/insertion de contenu de la source
  - Ajout de contenus nouveaux
  - Suppression (=ne pas utiliser certaines données)
  - Déplacement
  - Duplication

Exemple
Comment passer de ça

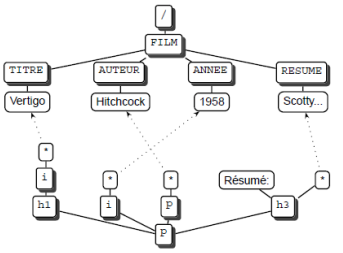
```
<?xml version="1.0" encoding="ISO-8859-1"?>
<SALLE NO='2' PLACES='120'>
   <FILM>
      <TITRE>Vertigo</TITRE>
      <AUTEUR>Alfred Hitchcock</AUTEUR>
      <ANNEE>1958</ANNEE>
      <RESUME>Scotty Ferguson, ancien inspecteur
      de police, est sujet au vertige depuis qu'il a vu
      mourir son collègue....
      </RESUME>
   </FILM>
</SALLE>
```


…à ceci (XML to HTML)

```
<p>
   <h1>
      <i>  Vertigo </i>
   </h1>
      <i>  1958  </i>
   <p>
      Hitchcock
   </p>
   <h3>Résumé:  Scotty…
   </h3>
</p>
```

Note:
-Pas tout le contenu.
-Des ajouts

Représentation en arbre



Ajout de contenus nouveaux

<xsl:text>le texte</xsl:text>

Génère le texte dans le fichier de sortie

Exemple:

<xsl:text> <p> <h1> </xsl:text>

<i> # La balise text est optionnelle

Lecture/insertion de contenu de la source

<xsl:value-of select="chemin XPath" />

- Lit le contenu au nœud indiqué (à partir du noeud courant = nœud de contexte)
- Exemple : à partir d'un noeud FILM, ajouter le titre :

  <xsl:value-of select="TITRE" />

  Lors de l'exécution, la valeur insérée sera le contenu de l'élément TITRE
  (eq. de Titre/text())

Exemple (suite)

Le programme de transformation

Note:
-On suppose: nœud courant = FILM
-La balise <xsl:text> est omise

```
<p>
   <h1>
        <i>  <xsl:value-of select="TITRE"/> </i>
   </h1>
        <i>  <xsl:value-of select="ANNEE"/>  </i>
   <p>
        <xsl:value-of select="AUTEUR"/>
   </p>
   <h3>Résumé:  <xsl:value-of select="RESUME"/>
   </h3>
</p>
```


Définir une règle de transformation

<xsl:template match="Chemin XPath">
… ici la règle à écrire
<xsl:template>

- template = définir la règle
- match = indique l'élément sur lequel s'applique la règle

- La règle sera utilisée avec un *apply-templates*

Définir une règle de transformation, exemple

Contenu fixe

```
<xsl:template match="Auteur"> Nom caché
</xsl:template> /*texte fixé pour tous les auteurs*/
```

Reproduire le titre du film dans une section <h1>

```
<xsl:template match="FILM">
   <h1> Le titre est
   <xsl:value-of select="TITRE"/>
   </h1>
</xsl:template>
```


Appliquer une règle de transformation

<xsl:apply-templates select="FILM" />

Applique la règle qui est définie, à partir de l'élément désigné (par select)

Exemple:

```
<xsl:template match="/">
   <xsl:apply-templates select="FILM/TITRE" />
   <xsl:apply-templates select="FILM/ANNEE" />
</xsl:template>
```

Note: Racine (début de l'analyse)

Note: ces 2 templates doivent être définis

Un programme de transformation

- C'est un ensemble de templates (=règles)
  - La règle initiale s'applique à la racine : '/'
  - L'application des règles s'enchaine et produit un résultat dans le document généré

Xpath.
Tiens, on s'est déjà vu!

Utilisé partout !
<xsl:apply-templates select="FILM/TITRE" />
<xsl:value-of select="FILM[1]/AUTEUR[last]" />

Rappel sur XPath
axe::filtre[prédicat1][prédicat2]…

-Axes Xpath: descendants, parents, frères...
-Filtres: accès aux éléments, attributs
-Prédicats: [1], [last], etc.